

Web Scraping

This are the step i am using in my project for Web Scraping using python library BeautifulSoup, requests and Pandas

1. Importing Required Libraries

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
```

- `requests` : Handles HTTP requests to fetch web pages
- `BeautifulSoup` : Parses HTML and XML documents to extract data
- `pandas` : Creates structured DataFrames for data manipulation and export

2. Setting Up the Request

```
url = '<https://en.wikipedia.org/wiki/List_of_largest_compani  
es_in_India>'

headers = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)  
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/91.0.4472.124 S  
afari/537.36'
}
```

- `url` : The Wikipedia page we want to scrape
- `headers` : Mimics a real browser request to avoid being blocked by Wikipedia's anti-bot measures
 - `User-Agent` tells the website we're using Chrome browser

3. Fetching the Web Page

```
response = requests.get(url, headers=headers)
soup = BeautifulSoup(response.content, 'html.parser')
```

- `requests.get()` : Sends a GET request to the URL with our headers
- `BeautifulSoup()` : Parses the HTML content into a structured format we can navigate
 - `'html.parser'` is Python's built-in HTML parser

4. Finding the Target Table

Debug Step

```
tables = soup.find_all('table')
print(f"Found {len(tables)} tables on the page")
```

- **Purpose:** Shows how many tables exist on the page (useful for debugging)
- Helps verify if the page loaded correctly

Three Methods to Find the Table

```
# Method 1: Find by heading text
heading = soup.find('span', {'id': '2025_Forbes_list'})
if heading:
    table = heading.find_parent().find_next('table')
```

- `soup.find('span', {'id': '2025_Forbes_list'})` : Finds the heading "2025 Forbes list"
- `heading.find_parent()` : Goes up to the parent element (usually a heading tag)
- `.find_next('table')` : Finds the first table immediately after this heading

```
# Method 2: Find by class name
else:
    table = soup.find('table', class_='wikitable')
```

- `find('table', class_='wikitable')`: Looks for a table with class "wikitable"
- Wikipedia tables typically have this class

```
# Method 3: Find by partial class match
if not table:
    table = soup.find('table', {'class': lambda c: c and 'wikitable' in c})
```

- `lambda c: c and 'wikitable' in c`: A function that checks if the class attribute contains "wikitable"
- More flexible than exact class matching

5. Extracting Data from the Table

Get All Rows

```
rows = table.find_all('tr')
```

- `find_all('tr')`: Finds all `<tr>` (table row) elements
- First row contains column headers, subsequent rows contain data

Extract Headers

```
header_row = rows[0]
headers = [header.get_text(strip=True) for header in header_row.find_all('th')]
```

- `rows[0]`: First row (header row)
- `find_all('th')`: Finds all `<th>` (table header) elements
- `get_text(strip=True)`: Extracts text and removes whitespace
- **List comprehension**: Creates a list of all column names

Extract Data Rows

```

data = []
for row in rows[1:]:
    cells = row.find_all('td')
    if len(cells) == len(headers):
        row_data = [cell.get_text(strip=True) for cell in cells]
        data.append(row_data)

```

- `rows[1:]`: Skip the header row, process all remaining rows
- `find_all('td')`: Finds all `<td>` (table data) elements
- `if len(cells) == len(headers)`: Only add rows that match header count
 - Prevents errors from merged cells or malformed rows
- **List comprehension**: Creates a list of cell values for each row

6. Creating and Displaying the DataFrame

```
df = pd.DataFrame(data, columns=headers)
```

- `pd.DataFrame()`: Converts the 2D data list into a pandas DataFrame
- `columns=headers`: Labels each column with the extracted headers

Display Results

```
print(df.head(10))
```

- `df.head(10)`: Shows first 10 rows of the DataFrame
- Useful for previewing the data

Save to CSV

```
df.to_csv('forbes_2025_india.csv', index=False)
```

- `to_csv()` : Exports DataFrame to CSV file
- `index=False` : Prevents pandas from adding an extra index column

7. Error Handling

```
else:
    print("✗ Table not found!")
    for i, tbl in enumerate(tables):
        print(f"Table {i}: class = {tbl.get('class', 'No class')}")
```

- If no table is found, prints all available table classes
- Helps debug why the table wasn't found